

Job Intelligence Platform (JEXA-AI / TalentLens)

Comprehensive Project Explanation

Project Overview:

An enterprise-grade, fully containerized microservices platform that scrapes, cleans, deduplicates, enriches, verifies, and indexes job listings - powered by ML-based salary estimation, scam detection, and semantic vector search.

Key Features:

- 8-Stage Real-Time Pipeline: Collector → Cleaner → Deduplicator → Enrichment → Verifier → Embedder → Notifier
- Web Scraping from 10+ sources: Adzuna, Indeed, LinkedIn, Naukri, government portals
- ML-Powered Intelligence: XGBoost scam detection, gradient boosting salary estimation, spaCy NER skill extraction
- Semantic Search: SentenceTransformers (all-MiniLM-L6-v2) → Qdrant vector database
- Full-Text Search: Elasticsearch 8 for keyword-based job search
- Production Observability: Prometheus metrics + Grafana dashboards + MLflow
- Two Frontend UIs: Public Job Board + Admin Dashboard
- Single-Command Deployment: docker compose up -d (19 containers)

Last Updated: 2026-09-02

Platform Version: 1.0.0

1. Project Structure

```
JEXA-AI/
├── services/                # Microservices (each has its own Dockerfile)
│   ├── gateway/            # FastAPI API gateway (:8000)
│   ├── collector/          # Job scraping from web sources
│   ├── cleaner/            # Data normalization + salary parsing
│   ├── deduplicator/       # Redis-based content fingerprinting
│   ├── enrichment/         # Skill extraction + salary estimation (ML)
│   ├── verifier/           # Scam detection + company trust scoring
│   ├── embedder/           # SentenceTransformers → Qdrant embeddings
│   ├── notifier/           # Email / webhook / in-app job alerts
│   ├── search/             # Elasticsearch semantic search
│   ├── analytics/          # Pipeline metrics aggregation
│   ├── auth/               # JWT authentication
│   ├── feedback/           # User feedback collection
│   ├── orchestrator/       # Pipeline orchestration
│   └── shared/             # Shared Python library
├── database/               # Motor/MongoDB session management
├── kafka/                  # Async Kafka consumer/producer
├── llm/                    # LLM router (OpenAI / Ollama)
├── models/                 # Pydantic models (Job, User, Company)
├── redis/                  # Redis client utilities
├── utils/                  # Logging, metrics, health checks
├── ml/                     # Machine learning modules
│   ├── salary_estimator/   # GradientBoosting salary prediction
│   ├── scam_detector/      # XGBoost scam detection
│   └── skill_extractor/    # spaCy NER skill pipeline
├── frontend/
│   ├── jobboard/           # Public job board UI (:3000)
│   └── admin/              # Admin dashboard UI (:3001)
├── infra/
│   ├── docker/            # Per-service Dockerfiles
│   ├── k8s/               # Kubernetes manifests
│   └── observability/     # Prometheus + Grafana config
├── tests/
│   ├── unit/              # Unit tests (no infrastructure needed)
│   └── integration/       # API tests (requires MongoDB)
├── scripts/               # Utility scripts
├── docker-compose.yml     # Development orchestration
├── docker-compose.prod.yml # Production orchestration
└── .env.example           # Environment variable template
```

2. Data Pipeline Architecture

Step	Action	Output Topic
1. Collector	Scrapes real jobs from web (Adzuna, Indeed, etc.)	raw_jobs
2. Cleaner	Strips HTML, parses salary/location	cleaned_jobs
3. Deduplicator	Redis fingerprint deduplication	deduplicated_jobs
4. Enrichment	ML skill extraction + salary estimation	enriched_jobs
5. Verifier	XGBoost scam detection + trust scoring	verified_jobs
6. Embedder	SentenceTransformer embeddings → Qdrant	published
7. Notifier	Sends alerts via email/webhook/in-app	notification_*
8. Gateway	Serves everything via REST API	Frontend UIs

All stages are connected via **Kafka topics** (powered by Redpanda). Each stage is an independent microservice that can be scaled, restarted, or replaced independently.

3. Docker Services Inventory

Infrastructure (11 Containers)

Container	Image	Port	Purpose
jip-mongodb	mongo:7.0	27017	Primary database
jip-redis	redis:7-alpine	6379	Cache + dedup fingerprints
jip-redpanda	redpandadata/redpanda	9092	Kafka-compatible broker
jip-redpanda-console	redpandadata/console	8080	Kafka management UI
jip-kafka-init	redpandadata/redpanda	—	Creates 24 Kafka topics
jip-qdrant	qdrant/qdrant	6333	Vector database (semantic search)
jip-elasticsearch	elasticsearch:8.17.0	9200	Full-text search engine
jip-minio	minio/minio	9000/9001	S3-compatible object storage
jip-prometheus	prom/prometheus	9090	Metrics collection
jip-grafana	grafana/grafana	3002	Monitoring dashboards
jip-mlflow	ghcr.io/mlflow/mlflow	5000	ML experiment tracking

Application (8 Microservices + 2 Frontends)

Container	Service	Type
jip-gateway	API Gateway	FastAPI HTTP (:8000)
jip-collector	Job Collector	Kafka Worker — scrapes real jobs
jip-cleaner	Data Cleaner	Kafka Worker — normalizes, strips HTML, parses salaries
jip-deduplicator	Deduplicator	Kafka Worker — Redis fingerprint-based dedup
jip-verifier	Verifier	Kafka Worker — ML scam detection + trust scoring
jip-enrichment	Enrichment	Kafka Worker — skill extraction + salary estimation
jip-embedder	Embedder	Kafka Worker — vector embeddings → Qdrant
jip-notifier	Notifier	Kafka Worker — email/webhook notifications
jip-jobboard	Job Board	Frontend UI (:3000)
jip-admin	Admin Dashboard	Frontend UI (:3001)

4. ML Models Summary

Model	Algorithm	Input	Output
Salary Estimator	Gradient Boosting + Rules	Title, skills, location, experience	Salary range (INR)
Scam Detector	XGBoost + Heuristics	Job text, URL, source metadata	Scam probability (0-1)
Skill Extractor	spaCy NER + Taxonomy	Job description text	Structured skill list (7 categories)

All models degrade gracefully to rule-based fallbacks when no trained .pkl file exists.

Training Commands:

```
# Salary estimator
python ml/salary_estimator/estimator.py --train --data data/labeled_salaries.csv

# Scam detector
python ml/scam_detector/train.py --data data/labeled_jobs.csv
```

5. Core Service Explanations

5.1 Gateway Service (*services/gateway/main.py*)

The FastAPI API Gateway is the single public entry point exposing REST APIs, Swagger docs, and rate limiting.

Key components:

- Module docstring and future annotations import (lines 1-12)
- Standard library imports for logging, environment, timing, and lifespan (lines 14-33)
- FastAPI, middleware, and rate-limiting imports (lines 35-42)
- Application routers import from services.gateway.routers (line 220)
- lifespan asyncontextmanager for startup/shutdown (lines 32-44) - inits MongoDB indexes in dev mode
- FastAPI app instantiation with title, description, docs, and lifespan (lines 47-95)
- Middleware: rate limiter, GZip compression, CORS (lines 99-119)
- Request timing middleware adding X-Process-Time-Ms header (lines 122-132)
- Router includes mounted at /api/v1/ (lines 135-147)
- Root, health, and readiness endpoints (lines 150-166) - health checks MongoDB and Redis

```
# Health check returns:
{
    "status": "healthy",
    "version": "1.0.0",
    "dependencies": {
        "mongodb": "healthy",
        "redis": "healthy"
    }
}
```

5.2 Collector Service (*services/collector/main.py*)

The CollectorService orchestrates all job collection across multiple web sources, publishing to Kafka.

Key components:

- Module docstring and imports (lines 1-33): asyncio, logging, env vars, Kafka clients
- SOURCE_WEIGHTS dict: proportional limit distribution across sources (lines 36-44)
- _scale_limits(): splits total fetch limit across sources (lines 46-54)
- CollectorService.__init__(): producer/consumer, locks, state (lines 56-72)
- start(): sets up clients, scheduled task, manual triggers (lines 74-95)
- _handle_trigger(): responds to manual Kafka triggers (lines 99-127)
- _scheduled_collection(): periodic collection every 90 min default (lines 129-152)
- _run_collection(): concurrent collection across all sources, batch publish (lines 154-205)
- main() entrypoint with logging setup (lines 235-246)

5.3 Collector Agents (*services/collector/agents.py*)

Provides concrete collectors for each source with BaseCollector handling rate limiting and circuit breaks.

Sources: Adzuna, Greenhouse, Lever, Workday, Indeed, Naukri, LinkedIn, RSS, Government, CompanyCareers, Manual

BaseCollector features:

- Async HTTP session with httpx.AsyncClient
- Semaphore-based concurrency control per source
- Token-bucket rate limiting with adaptive multiplier on 429/503
- Header rotation to reduce fingerprinting
- Circuit breaker to avoid hammering failing sources
- _rate_limited_get() centralizes request timing and circuit breaker updates
- retry_with_backoff() with exponential backoff + jitter, honors Retry-After

Note: Some collectors (Naukri, Workday, LinkedIn) require credentials/captcha solvers to function.

5.4 Deduplicator Service (*services/deduplicator/main.py*)

Consumes cleaned jobs, performs Redis fingerprint-based deduplication, publishes unique jobs downstream.

Key functions:

- `generate_fingerprint()`: SHA-256 hash of normalized title+company+location
- `DeduplicatorService._handle_batch()`: core logic (lines 91-220)
 - Precompute fingerprints and source IDs
 - Single DB query for exact duplicates by source+source_job_id
 - Second query for fingerprint matches
 - Classifies: exact dup, near dup, batch-internal dup, or unique
 - Bulk upserts for jobs (mark `is_duplicate`, `duplicate_of_id`, `status`)
 - Publishes unique messages to `TOPICS.JOB_DEDUPLICATED`

5.5 Enrichment Service (*services/enrichment/main.py*)

Consumes deduplicated jobs, enriches with skills, salary, tech stack, and domain tags.

Key components:

- `EnrichmentService.start()` (lines 40-88): inits `SkillExtractionAgent`, `SalaryExtractionAgent`, optionally `SalaryEstimator`
- `_enrich_one()` (lines 90-168): enriches single job
 - Extracts title, description, company, salary fields
 - Runs skill extraction and salary extraction concurrently (`asyncio.create_task` + `gather`)
 - Constructs required/optional skills, tech stack, domain tags
 - Attempts ML salary estimation via `_build_salary_data()`
 - Updates message with enrichment fields
- `_handle_batch()` (lines 170-246): semaphore-limited concurrent enrichment, bulk writes
- `_build_salary_data()` (lines 248-314): merges LLM/rule output with ML estimator, picks higher confidence, falls back to default

5.6 Verifier Service (*services/verifier/main.py*)

Consumes enriched jobs, performs ML-based scam detection and trust scoring, routes to verified/rejected.

Key components:

- `VerifierService.start()` (lines 42-88): loads `ScamDetectionAgent`, starts Kafka clients
 - `_verify_one()` (lines 90-139): runs scam analysis, computes verification metadata
 - Calls `scam_agent.analyze()` to get scam probability
 - Computes `quality_score` via `_compute_quality_score()`
 - Determines `is_rejected` if `scam_probability` $\geq$ `SCAM_THRESHOLD` (default 0.7)
 - `_handle_batch()` (lines 141-233): parallel verification with semaphore
 - `_compute_quality_score()` (lines 235-258): heuristic scoring
 - Base score: 50.0
 - +15 if description > 500 chars
 - +15 if $\geq$ 5 skills
 - +10 if both `salary_min` and `salary_max` present
 - - `scam_probability` * 40 (penalizes scam risk)
 - Clamped to [0, 100]
- ```
Verification metadata fields:
scam_probability, scam_risk_level, scam_triggered_rules,
authenticity_score (100 - scam_prob*100), quality_score,
is_verified (< 0.2 scam prob), status (verified/rejected)
```

## 5.7 Embedder Service (*services/embedder/main.py*)

Consumes verified jobs, generates SentenceTransformer embeddings, indexes in Qdrant, updates MongoDB.

Key components:

- Configuration constants (lines 22-33): model, Qdrant host/port/collection, vector size 384, batch size
- EmbedderService.start() (lines 48-76): loads SentenceTransformer model, inits Qdrant collection
- \_handle\_batch() (lines 78-224): core processing
  - Build embed\_texts from title+description+company+location+skills
  - Run model.encode() in thread executor (avoid blocking event loop)
  - Upsert vectors to Qdrant with UUIDs and payload metadata
  - Bulk MongoDB updates to set embedding\_id, embedding\_model, status
  - Inserts pipeline event records
  - Fallback to concurrent per-job updates if bulk\_write fails
  - Logs performance metrics (jobs/sec)

## 5.8 Kafka Clients

shared/kafka/producer.py:

- KafkaProducerClient wraps aiokafka.AIOKafkaProducer with JSON serialization
- send\_batch(): publishes many messages in parallel, bounds concurrency
- Normalizes to (payload, key) tuples, uses sub-batches
- Failed records forwarded to dead-letter queue (.dlq)
- acks='all', enable\_idempotence=True, LZ4 compression

shared/kafka/consumer.py:

- KafkaConsumerClient wraps aiokafka.AIOKafkaConsumer
- consume(): single-item handler mode
- consume\_batch(): high-throughput batch mode using getmany()
- Manual commits (enable\_auto\_commit=False) for safe offset control

## 6. Database Models (shared/database/models.py)

Pydantic models defining MongoDB document schemas used across the pipeline.

Key models:

| Model           | Purpose                                                    | Key Fields                                      |
|-----------------|------------------------------------------------------------|-------------------------------------------------|
| MongoBaseModel  | Base model with _id alias, UTC timestamps, extra='allow'   | _id, created_at, updated_at                     |
| Company         | Company schema with name, normalized_name, trust indicator | name, normalized_name, ...                      |
| Job             | Main job schema with 40+ fields for pipeline processing    | source, title, description, salary, skills, ... |
| User            | User account schema                                        | email, hashedPassword, role, isActive           |
| UserProfile     | Candidate profile with resume data                         | firstName, lastName, skills, experience         |
| SavedSearch     | User's saved search criteria                               | query, filters                                  |
| SavedJob        | User bookmarking a job                                     | userId, jobId, savedAt                          |
| PipelineEvent   | Audit log for job pipeline stages                          | jobId, stage, status, timestamp                 |
| NotificationLog | Sent notification records                                  | userId, type, message, read                     |
| ScamRule        | Configurable scam detection rules                          | pattern, severity, category                     |
| SkillTaxonomy   | Skill categorization (7 categories)                        | name, category, aliases                         |
| CollectionRun   | Collection metadata and stats                              | source, runTime, count, status                  |

Database session (shared/database/session.py):

- `get_mongo_client()`: lazy singleton Motor client (pool: min 5, max 50, timeouts 5s)
- `get_db()`: FastAPI dependency yielding Motor DB handle
- `create_tables()`: initializes MongoDB indexes via `shared.database.base.create_all_indexes`
- `close_client()`: graceful shutdown
- `drop_tables()`: dev/test helper (env-gated)

## 7. Observability & Monitoring

| Tool             | URL                             | Purpose                                  |
|------------------|---------------------------------|------------------------------------------|
| Prometheus       | http://localhost:9090           | Scrapes all services at /metrics         |
| Grafana          | http://localhost:3002           | Pre-built dashboards (admin/admin)       |
| Redpanda Console | http://localhost:8080           | Kafka topic inspection & consumer groups |
| MLflow           | http://localhost:5000           | ML experiment tracking & model registry  |
| Qdrant Dashboard | http://localhost:6333/dashboard | Vector database management               |

### Quick Deployment Commands:

```
Start everything
docker compose up -d

Rebuild after code changes
docker compose up -d --build

View logs
docker compose logs -f <service>

Stop (preserve data)
docker compose stop

Full wipe
docker compose down -v

Health check
curl http://localhost:8000/health
```

---

End of Project Explanation Document